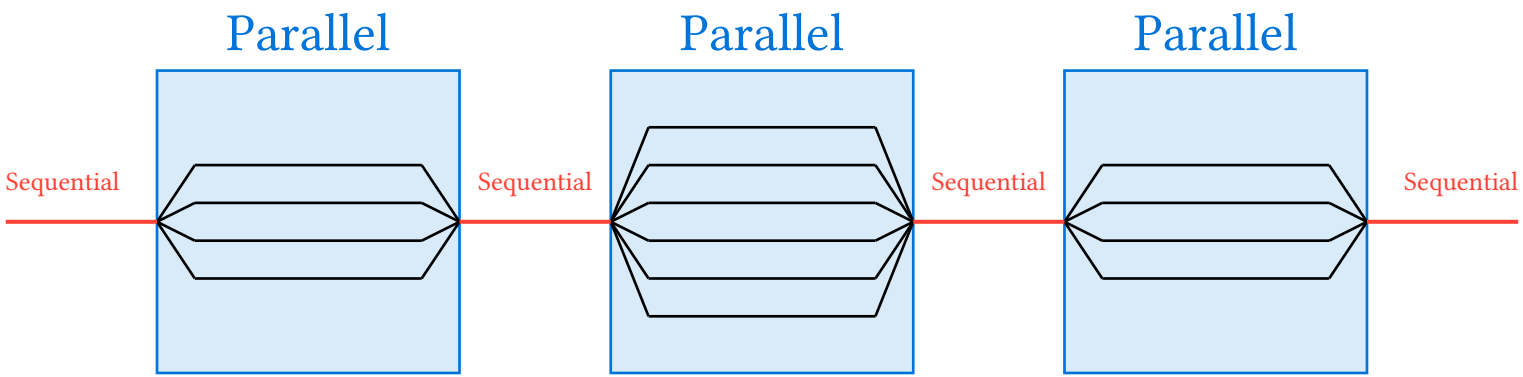
Introduction to OpenMP

Parallelism TP2

Recall & principle



OpenMP approach: specify a parallel section using `#pragma` directives

Check the documentation for the different clauses

Basic scope

```
omp_set_num_threads(4);

#pragma omp parallel
{
    std::cout << "Hello from " << omp_get_thread_num() << std::endl;
}
```

`omp_set_num_threads`: set the number of threads to use

`omp_get_thread_num`: get the id of current thread

OpenMP: Loop

For loop parallelisation, `#pragma omp parallel` is heavy:

- Use `omp_set_num_threads` to fix the number of threads
- Use `omp_get_thread_num` as index
- Far from the for loop structure

```
std::vector<int> data(10, 0);

#pragma omp parallel for
for(size_t i = 0; i < data.size(); ++i) {
  // Work on data...
}
```

`#pragma omp for`: Allows automatic assignment of indices to different threads

OpenMP: Variable visibility

Three types of visibility:

- shared
- private
- firstprivate

```
std::vector<int> data(10, 0); // shared (default)
unsigned seed = 42;

#pragma omp parallel for firstprivate(seed)
for(size_t i = 0; i < data.size(); ++i) {
  int x = 2; // thread-local
  seed *= i;
  data[i] = seed;
}
```

OpenMP: Reduction

Goal: sum the values of a vector

```
std::vector<int> data(10, 0);
int sum = 0; // shared

#pragma omp parallel for
for(size_t i = 0; i < data.size(); ++i) {
  sum += data[i];
}
```

Problem: race condition on `sum`

OpenMP: Reduction

Solution:

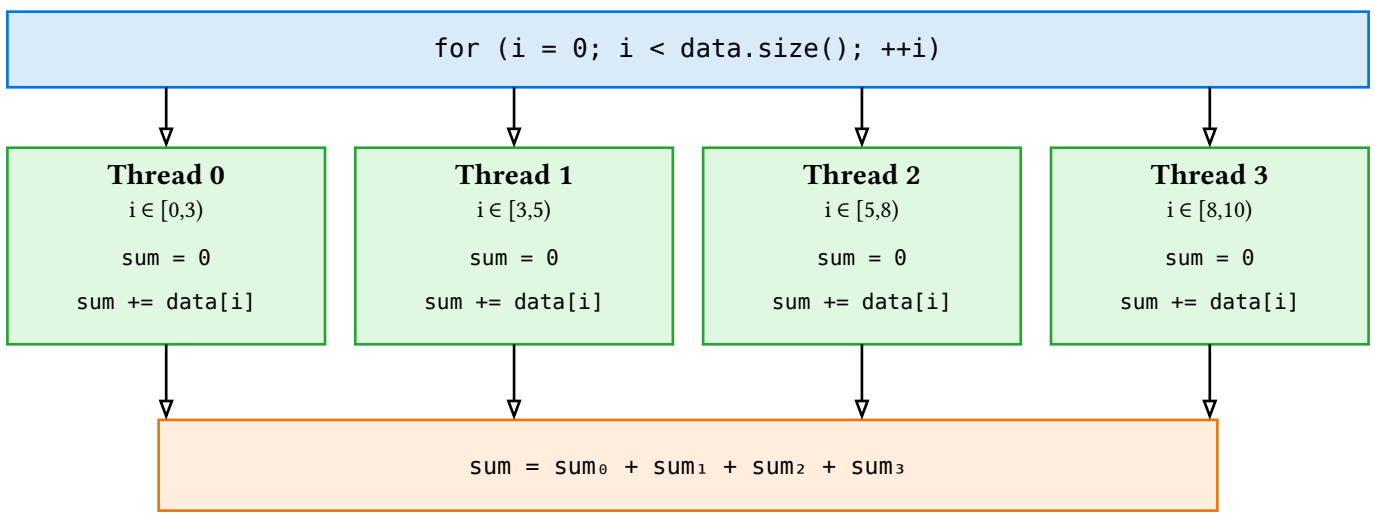
```
std::vector<int> data(10, 0);
int sum = 0;

#pragma omp parallel for reduction(+:sum)
for(size_t i = 0; i < data.size(); ++i) {
  sum += data[i];
}
```

`reduction(op:var)`:

- `var`: reduction variable, defined as private
- `op`: reduction operator (defines also the initial value of `var`)

OpenMP: Reduction



OpenMP: Compilation

OpenMP support is built into the compiler but requires a flag to be enabled during compilation:

```
g++ main.cpp -o main -fopenmp
```